Experiment 'p_13_3_recursive_reverse_string' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_13_3_recursive_reverse_string
Natural language statement: Write a recursive method that takes a string and returns the string in reverse. For example, reverse("Hello!") returns the string "!olleH". Implement a recursive solution by removing the first character, reversing the remaining text, and combining the two.
Method signature: p_13_3_recursive_reverse_string(text: string) returns (reversed: string)

Ensures

- `|reversed| == |text|`
- `forall i :: 0 <= i < |text| ==> reversed[i] == text[|text| - 1 - i]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a recursive method that takes a string and returns the string in reverse. For
    ↪ example, reverse("Hello!") returns the string "!olleH". Implement a recursive
    ↪ solution by removing the first character, reversing the remaining text, and
    ↪ combining the two.

The signature should be:

p_13_3_recursive_reverse_string(text: string) returns (reversed: string)

The method should respect the following contract:

ensures |reversed| == |text|, ensures forall i :: 0 <= i < |text| ==> reversed[i] == text
    ↪ [|text| - 1 - i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnAX7wrJvaaPKFlPcqWGwRpJgHWdR
Model: gpt-5.5-2026-04-23
Created at: 1780610933
Finish reason: stop
Usage: 390 tokens in, and 2458 tokens out

```
{
  if |text| == 0 {
    reversed := "";
  } else {
    var rest := text[1..];
    var restReversed := p_13_3_recursive_reverse_string(rest);
    reversed := restReversed + text[0..1];

    assert |rest| == |text| - 1;
    assert |text[0..1]| == 1;
    assert |restReversed| == |rest|;
    assert |reversed| == |text|;

    forall i | 0 <= i < |text|
      ensures reversed[i] == text[|text| - 1 - i]
    {
      if i < |text| - 1 {
```

```
      assert i < |restReversed|;
      assert reversed[i] == restReversed[i];
      assert restReversed[i] == rest[|rest| - 1 - i];
      assert 0 <= |rest| - 1 - i < |rest|;
      assert rest[|rest| - 1 - i] == text[1 + (|rest| - 1 - i)];
      assert 1 + (|rest| - 1 - i) == |text| - 1 - i;
    } else {
      assert i == |text| - 1;
      assert i == |restReversed|;
      assert reversed[i] == text[0..1][0];
      assert text[0..1][0] == text[0];
      assert |text| - 1 - i == 0;
    }
  }
 }
}
```

Final Program

```
method p_13_3_recursive_reverse_string(text: string) returns (reversed: string)
      ensures |reversed| == |text|
      ensures forall i :: 0 <= i < |text| ==> reversed[i] == text[|text| - 1 - i]
{
 if |text| == 0 {
  reversed := "";
 } else {
  var rest := text[1..];
  var restReversed := p_13_3_recursive_reverse_string(rest);
  reversed := restReversed + text[0..1];

  assert |rest| == |text| - 1;
  assert |text[0..1]| == 1;
  assert |restReversed| == |rest|;
  assert |reversed| == |text|;

  forall i | 0 <= i < |text|
    ensures reversed[i] == text[|text| - 1 - i]
  {
    if i < |text| - 1 {
      assert i < |restReversed|;
      assert reversed[i] == restReversed[i];
      assert restReversed[i] == rest[|rest| - 1 - i];
      assert 0 <= |rest| - 1 - i < |rest|;
      assert rest[|rest| - 1 - i] == text[1 + (|rest| - 1 - i)];
      assert 1 + (|rest| - 1 - i) == |text| - 1 - i;
    } else {
      assert i == |text| - 1;
      assert i == |restReversed|;
      assert reversed[i] == text[0..1][0];
      assert text[0..1][0] == text[0];
```

```
      assert |text| - 1 - i == 0;
    }
   }
  }
}
```

Total Token Usage

Input tokens: 390
Output tokens: 2458
Reasoning tokens: 2047
Sum of 'total tokens': 2848

Experiment Timings

Overall Experiment started at 1780610926472, ended at 1780611014181, lasting 87709ms (87.71 seconds)
Iteration #1 started at 1780610926486, ended at 1780611014178, lasting 87692ms (87.69 seconds)